

表 17.13: 匹配标志

744

定义在 <code>regex_constants::match_flag_type</code> 中	
<code>match_default</code>	等价于 <code>format_default</code>
<code>match_not_bol</code>	不将首字符作为行首处理
<code>match_not_eol</code>	不将尾字符作为行尾处理
<code>match_not_bow</code>	不将首字符作为单词首处理
<code>match_not_eow</code>	不将尾字符作为单词尾处理
<code>match_any</code>	如果存在多于一个匹配, 则可返回任意一个匹配
<code>match_not_null</code>	不匹配任何空序列
<code>match_continuous</code>	匹配必须从输入的首字符开始
<code>match_prev_avail</code>	输入序列包含第一个匹配之前的内容
<code>format_default</code>	用 ECMAScript 规则替换字符串
<code>format_sed</code>	用 POSIX sed 规则替换字符串
<code>format_no_copy</code>	不输出输入序列中未匹配的部分
<code>format_first_only</code>	只替换子表达式的第一次出现

使用格式标志

默认情况下, `regex_replace` 输出整个输入序列。未与正则表达式匹配的部分会原样输出; 匹配的部分按格式字符串指定的格式输出。我们可以通过在 `regex_replace` 调用中指定 `format_no_copy` 来改变这种默认行为:

```
// 只生成电话号码: 使用新的格式字符串
string fmt2 = "$2.$5.$7 "; // 在最后一部分号码后放置空格作为分隔符
// 通知 regex_replace 只拷贝它替换的文本
cout << regex_replace(s, r, fmt2, format_no_copy) << endl;
```

给定相同的输入, 此版本的程序生成

```
201.555.2368 862.555.0123
973.555.0130
609.555.0132 201.555.0175 800.555.0000
```

17.3.4 节练习

- 练习 17.24: 编写你自己版本的重排电话号码格式的程序。
- 练习 17.25: 重写你的电话号码程序, 使之只输出每个人的第一个电话号码。
- 练习 17.26: 重写你的电话号码程序, 使之对多于一个电话号码的人只输出第二个和后续电话号码。
- 练习 17.27: 编写程序, 将九位数字邮政编码的格式转换为 dddd-dddd。

17.4 随机数

745

程序通常需要一个随机数源。在新标准出现之前, C 和 C++ 都依赖于一个简单的 C 库函数 `rand` 来生成随机数。此函数生成均匀分布的伪随机整数, 每个随机数的范围在 0 和一个系统相关的最大值 (至少为 32767) 之间。

C++ 11

rand 函数有一些问题：即使不是大多数，也有很多程序需要不同范围的随机数。一些应用需要随机浮点数。一些程序需要非均匀分布的数。而程序员为了解决这些问题而试图转换 rand 生成的随机数的范围、类型或分布时，常常会引入非随机性。

定义在头文件 random 中的随机数库通过一组协作的类来解决这些问题：随机数引擎类（random-number engines）和随机数分布类（random-number distribution）。表 17.14 描述了这些类。一个引擎类可以生成 unsigned 随机数序列，一个分布类使用一个引擎类生成指定类型的、在给定量范围内的、服从特定概率分布的随机数。

表 17.14：随机数库的组成	
引擎	类型，生成随机 unsigned 整数序列
分布	类型，使用引擎返回服从特定概率分布的随机数

Best Practices

C++程序不应该使用库函数 rand，而应使用 default_random_engine 类和恰当的分布类对象。

17.4.1 随机数引擎和分布

随机数引擎是函数对象类（参见 14.8 节，第 506 页），它们定义了一个调用运算符，该运算符不接受参数并返回一个随机 unsigned 整数。我们可以通过调用一个随机数引擎对象来生成原始随机数：

```
default_random_engine e; // 生成随机无符号数
for (size_t i = 0; i < 10; ++i)
    // e() “调用” 对象来生成下一个随机数
    cout << e() << " ";
```

在我们的系统中，此程序生成：

```
16807 282475249 1622650073 984943658 1144108930 470211272 ...
```

在本例中，我们定义了一个名为 e 的 default_random_engine 对象。在 for 循环内，我们调用对象 e 来获得下一个随机数。

746

标准库定义了多个随机数引擎类，区别在于性能和随机性质量不同。每个编译器都会指定其中一个作为 default_random_engine 类型。此类型一般具有最常用的特性。表 17.15 列出了随机数引擎操作，标准库定义的引擎类型列在附录 A.3.2（第 783 页）中。

表 17.15：随机数引擎操作	
Engine e;	默认构造函数；使用该引擎类型默认的种子
Engine e(s);	使用整型值 s 作为种子
e.seed(s)	使用种子 s 重置引擎的状态
e.min()	此引擎可生成的最小值和最大值
e.max()	
Engine::result_type	此引擎生成的 unsigned 整型类型
e.discard(u)	将引擎推进 u 步；u 的类型为 unsigned long long

对于大多数场合，随机数引擎的输出是不能直接使用的，这也是为什么早先我们称之为原始随机数。问题出在生成的随机数的值范围通常与我们需要的不符，而正确转换随机数的范围是极其困难的。

分布类型和引擎

为了得到在一个指定范围内的数，我们使用一个分布类型的对象：

```
// 生成 0 到 9 之间（包含）均匀分布的随机数
uniform_int_distribution<unsigned> u(0,9);
default_random_engine e; // 生成无符号随机整数
for (size_t i = 0; i < 10; ++i)
    // 将 u 作为随机数源
    // 每个调用返回在指定范围内并服从均匀分布的值
    cout << u(e) << " ";
```

此代码生成下面这样的输出

0 1 7 4 5 2 0 6 6 9

此处我们将 `u` 定义为 `uniform_int_distribution<unsigned>`。此类型生成均匀分布的 `unsigned` 值。当我们定义一个这种类型的对象时，可以提供想要的最小值和最大值。在此程序中，`u(0,9)` 表示我们希望得到 0 到 9 之间（包含）的数。随机数分布类会使用包含的范围，从而我们可以得到给定整型类型的每个可能值。

类似引擎类型，分布类型也是函数对象类。分布类型定义了一个调用运算符，它接受一个随机数引擎作为参数。分布对象使用它的引擎参数生成随机数，并将其映射到指定的分布。

注意，我们传递给分布对象的是引擎对象本身，即 `u(e)`。如果我们将调用写成 `u(e())`，含义就变为将 `e` 生成的下一个值传递给 `u`，会导致一个编译错误。我们传递的是引擎本身，而不是它生成的下一个值，原因是某些分布可能需要调用引擎多次才能得到一个值。



当我们说随机数发生器时，是指分布对象和引擎对象的组合。

比较随机数引擎和 rand 函数

对熟悉 C 库函数 `rand` 的读者，值得注意的是：调用一个 `default_random_engine` 对象的输出类似 `rand` 的输出。随机数引擎生成的 `unsigned` 整数在一个系统定义的范围，而 `rand` 生成的数的范围在 0 到 `RAND_MAX` 之间。一个引擎类型的范围可以通过调用该类型对象的 `min` 和 `max` 成员来获得：

```
cout << "min: " << e.min() << " max: " << e.max() << endl;
```

在我们的系统中，此程序生成下面的输出：

min: 1 max: 2147483646

引擎生成一个数值序列

随机数发生器有一个特性经常会使新手迷惑：即使生成的数看起来是随机的，但对一个给定的发生器，每次运行程序它都会返回相同的数值序列。序列不变这一事实在调试时非常有用。但另一方面，使用随机数发生器的程序也必须考虑这一特性。

作为一个例子，假定我们需要一个函数生成一个 `vector`，包含 100 个均匀分布在 0 到 9 之间的随机数。我们可能认为应该这样编写此函数：

```
// 几乎肯定是生成随机整数 vector 的错误方法
// 每次调用这个函数都会生成相同的 100 个数!
vector<unsigned> bad_randVec()
{
    default_random_engine e;
    uniform_int_distribution<unsigned> u(0,9);
    vector<unsigned> ret;
    for (size_t i = 0; i < 100; ++i)
        ret.push_back(u(e));
    return ret;
}
```

但是，每次调用这个函数都会返回相同的 vector：

```
vector<unsigned> v1(bad_randVec());
vector<unsigned> v2(bad_randVec());
// 将打印"equal"
cout << ((v1 == v2) ? "equal" : "not equal") << endl;
```

748 此代码会打印 equal，因为 vector v1 和 v2 具有相同的值。

编写此函数的正确方法是将引擎和关联的分布对象定义为 static 的（参见 6.1.1 节，第 185 页）：

```
// 返回一个 vector，包含 100 个均匀分布的随机数
vector<unsigned> good_randVec()
{
    // 由于我们希望引擎和分布对象保持状态，因此应该将它们
    // 定义为 static 的，从而每次调用都生成新的数
    static default_random_engine e;
    static uniform_int_distribution<unsigned> u(0,9);
    vector<unsigned> ret;
    for (size_t i = 0; i < 100; ++i)
        ret.push_back(u(e));
    return ret;
}
```

由于 e 和 u 是 static 的，因此它们在函数调用之间会保持住状态。第一次调用会使用 u(e) 生成的序列中的前 100 个随机数，第二次调用会获得接下来 100 个，依此类推。



一个给定的随机数发生器一直会生成相同的随机数序列。一个函数如果定义了局部的随机数发生器，应该将其（包括引擎和分布对象）定义为 static 的。否则，每次调用函数都会生成相同的序列。

设置随机数发生器种子

随机数发生器会生成相同的随机数序列这一特性在调试中很有用。但是，一旦我们的程序调试完毕，我们通常希望每次运行程序都会生成不同的随机结果，可以通过提供一个种子（seed）来达到这一目的。种子就是一个数值，引擎可以利用它从序列中一个新位置重新开始生成随机数。

为引擎设置种子有两种方式：在创建引擎对象时提供种子，或者调用引擎的 seed 成员：

```

default_random_engine e1;                // 使用默认种子
default_random_engine e2(2147483646);    // 使用给定的种子值
// e3 和 e4 将生成相同的序列，因为它们使用了相同的种子
default_random_engine e3;                // 使用默认种子值
e3.seed(32767);                          // 调用 seed 设置一个新种子值
default_random_engine e4(32767);        // 将种子值设置为 32767
for (size_t i = 0; i != 100; ++i) {
    if (e1() == e2())
        cout << "unseeded match at iteration: " << i << endl;
    if (e3() != e4())
        cout << "seeded differs at iteration: " << i << endl;
}

```

本例中我们定义了四个引擎。前两个引擎 e1 和 e2 的种子不同，因此应该生成不同的序列。后两个引擎 e3 和 e4 有相同的种子，它们将生成相同的序列。

749

选择一个好的种子，与生成好的随机数所涉及的其他大多数事情相同，是极其困难的。可能最常用的方法是调用系统函数 time。这个函数定义在头文件 ctime 中，它返回从一个特定时刻到当前经过了多少秒。函数 time 接受单个指针参数，它指向用于写入时间的数据结构。如果此指针为空，则函数简单地返回时间：

```
default_random_engine e1(time(0)); // 稍微随机些的种子
```

由于 time 返回以秒计的时间，因此这种方式只适用于生成种子的间隔为秒级或更长的应用。



如果程序作为一个自动过程的一部分反复运行，将 time 的返回值作为种子的方式就无效了；它可能多次使用的都是相同的种子。

17.4.1 节练习

练习 17.28: 编写函数，每次调用生成并返回一个均匀分布的随机 unsigned int。

练习 17.29: 修改上一题中编写的函数，允许用户提供一个种子作为可选参数。

练习 17.30: 再次修改你的程序，此次再增加两个参数，表示函数允许返回的最小值和最大值。

17.4.2 其他随机数分布

随机数引擎生成 unsigned 数，范围内的每个数被生成的概率都是相同的。而应用程序常常需要不同类型或不同分布的随机数。标准库通过定义不同随机数分布对象来满足这两方面的要求，分布对象和引擎对象协同工作，生成要求的结果。表 17.16 列出了分布类型所支持的操作。

生成随机实数

程序常需要一个随机浮点数的源。特别是，程序经常需要 0 到 1 之间的随机数。

最常用但不正确的从 rand 获得一个随机浮点数的方法是用 rand() 的结果除以 RAND_MAX，即，系统定义的 rand 可以生成的最大随机数的上界。这种方法不正确的原因是随机整数的精度通常低于随机浮点数，这样，有一些浮点值就永远不会被生成了。

使用新标准库设施，可以很容易地获得随机浮点数。我们可以定义一个

750

`uniform_real_distribution` 类型的对象, 并让标准库来处理从随机整数到随机浮点数的映射。与处理 `uniform_int_distribution` 一样, 在定义对象时, 我们指定最小值和最大值:

```
default_random_engine e; // 生成无符号随机整数
// 0 到 1 (包含) 的均匀分布
uniform_real_distribution<double> u(0,1);
for (size_t i = 0; i < 10; ++i)
    cout << u(e) << " ";
```

这段代码与之前生成 `unsigned` 值的程序几乎相同。但是, 由于我们使用了一个不同的分布类型, 此版本会生成不同的结果:

```
0.131538 0.45865 0.218959 0.678865 0.934693 0.519416 ...
```

表 17.16: 分布类型的操作	
<code>Dist d;</code>	默认构造函数: 使 <code>d</code> 准备好被使用。 其他构造函数依赖于 <code>Dist</code> 的类型; 参见附录 A.3 节 (第 781 页)。 分布类型的构造函数是 <code>explicit</code> 的 (参见 7.5.4 节, 第 265 页)
<code>d(e)</code>	用相同的 <code>e</code> 连续调用 <code>d</code> 的话, 会根据 <code>d</code> 的分布式类型生成一个随机数序列; <code>e</code> 是一个随机数引擎对象
<code>d.min()</code>	返回 <code>d(e)</code> 能生成的最小值和最大值
<code>d.max()</code>	
<code>d.reset()</code>	重建 <code>d</code> 的状态, 使得随后对 <code>d</code> 的使用不依赖于 <code>d</code> 已经生成的值

使用分布的默认结果类型

分布类型都是模板, 具有单一的模板类型参数, 表示分布生成的随机数的类型, 对此有一个例外, 我们将在 17.4.2 节 (第 665 页) 中进行介绍。这些分布类型要么生成浮点类型, 要么生成整数类型。

每个分布模板都有一个默认模板实参 (参见 16.1.3 节, 第 594 页)。生成浮点值的分布类型默认生成 `double` 值, 而生成整型值的分布默认生成 `int` 值。由于分布类型只有一个模板参数, 因此当我们希望使用默认随机数类型时要记得在模板名之后使用空尖括号 (参见 16.1.3 节, 第 594 页):

```
// 空<>表示我们希望使用默认结果类型
uniform_real_distribution<> u(0,1); // 默认生成 double 值
```

751 生成非均匀分布的随机数

除了正确生成在指定范围内的数之外, 新标准库的另一个优势是可以生成非均匀分布的随机数。实际上, 新标准库定义了 20 种分布类型, 这些类型列在附录 A.3 (第 781) 中。

作为一个例子, 我们将生成一个正态分布的值的序列, 并画出值的分布。由于 `normal_distribution` 生成浮点值, 我们的程序使用头文件 `cmath` 中的 `lround` 函数将每个随机数舍入到最接近的整数。我们将生成 200 个数, 它们以均值 4 为中心, 标准差为 1.5。由于使用的是正态分布, 我们期望生成的数中大约 99% 都在 0 到 8 之间 (包含)。我们的程序会对这个范围内的每个整数统计有多少个生成的数映射到它:

```
default_random_engine e; // 生成随机整数
normal_distribution<> n(4,1.5); // 均值 4, 标准差 1.5
```

```
vector<unsigned> vals(9);           // 9个元素均为0
for (size_t i = 0; i != 200; ++i) {
    unsigned v = lround(n(e));      // 舍入到最接近的整数
    if (v < vals.size())            // 如果结果在范围内
        ++vals[v];                 // 统计每个数出现了多少次
}
for (size_t j = 0; j != vals.size(); ++j)
    cout << j << ": " << string(vals[j], '*') << endl;
```

我们首先定义了随机数发生器对象和一个名为 `vals` 的 `vector`。我们用 `vals` 来统计范围 0...8 中的每个数出现了多少次。与我们使用 `vector` 的大多数程序不同，此程序按需求大小为 `vals` 分配空间，每个元素都被初始化为 0。

在 `for` 循环中，我们调用 `lround(n(e))` 来将 `n(e)` 返回的值舍入到最接近的整数。获得浮点随机数对应的整数后，我们将它作为计数器 `vector` 的下标。由于 `n(e)` 可能生成范围 0 到 8 之外的数，所以我们首先检查生成的数是否在范围内，然后再将其作为 `vals` 的下标。如果结果确实在范围内，我们递增对应的计数器。

当循环结束时，我们打印 `vals` 的内容，可能会打印出像下面这样的结果：

```
0: ***
1: *****
2: *****
3: *****
4: *****
5: *****
6: *****
7: *****
8: *
```

本例中我们打印一个由星号组成的 `string`，有多少随机数等于此下标我们就打印多少个星号。注意，此图并不是完美对称的。如果打印出的图是完美对称的，我们反倒有理由怀疑随机数发生器的质量了。

752

bernoulli_distribution 类

我们注意到有一个分布不接受模板参数，即 `bernoulli_distribution`，因为它是一个普通类，而非模板。此分布总是返回一个 `bool` 值。它返回 `true` 的概率是一个常数，此概率的默认值是 0.5。

作为一个这种分布的例子，我们可以编写一个程序，这个程序与用户玩一个游戏。为了进行这个游戏，其中一个游戏者——用户或是程序——必须先行。我们可以用一个值范围是 0 到 1 的 `uniform_int_distribution` 来选择先行的游戏者，但也可以用伯努利分布来完成这个选择。假定已有一个名为 `play` 的函数来进行游戏，我们可以编写像下面这样的循环来与用户交互：

```
string resp;
default_random_engine e; // e 应保持状态，所以必须在循环外定义！
bernoulli_distribution b; // 默认是 50/50 的机会
do {
    bool first = b(e);    // 如果为 true，则程序先行
    cout << (first ? "We go first"
                : "You get to go first") << endl;
```

```
// 传递谁先行的指示, 进行游戏
cout << ((play(first)) ? "sorry, you lost"
        : "congrats, you won") << endl;
cout << "play again? Enter 'yes' or 'no'" << endl;
} while (cin >> resp && resp[0] == 'y');
```

我们用 `do while` 循环 (参见 5.4.4 节, 第 169 页) 来反复提示用户进行游戏。



由于引擎返回相同的随机数序列 (参见 17.4.1 节, 第 661 页), 所以我们在循环外声明引擎对象。否则, 每步循环都会创建一个新引擎, 从而每步循环都会生成相同的值。类似的, 分布对象也要保持状态, 因此也应该在循环外定义。

在此程序中使用 `bernoulli_distribution` 的一个原因是它允许我们调整选择先行一方的概率:

```
bernoulli_distribution b(.55); // 给程序一个微小的优势
```

如果 `b` 定义如上, 则程序有 55/45 的机会先行。

17.4.2 节练习

练习 17.31: 对于本节中的游戏程序, 如果在 `do` 循环内定义 `b` 和 `e`, 会发生什么?

练习 17.32: 如果我们在循环内定义 `resp`, 会发生什么?

练习 17.33: 修改 11.3.6 节 (第 392 页) 中的单词转换程序, 允许对一个给定单词有多种转换方式, 每次随机选择一种进行实际转换。

17.5 IO 库再探

在第 8 章中我们介绍了 IO 库的基本结构及其最常用的部分。在本节中, 我们将介绍三个更特殊的 IO 库特性: 格式控制、未格式化 IO 和随机访问。

753 17.5.1 格式化输入与输出

除了条件状态外 (参见 8.1.2 节, 第 279 页), 每个 `iostream` 对象还维护一个格式状态来控制 IO 如何格式化的细节。格式状态控制格式化的某些方面, 如整型值是几进制、浮点值的精度、一个输出元素的宽度等。

标准库定义了一组操纵符 (`manipulator`) (参见 1.2 节, 第 6 页) 来修改流的格式状态, 如表 17.7 和表 17.8 所示。一个操纵符是一个函数或是一个对象, 会影响流的状态, 并能用作输入或输出运算符的运算对象。类似输入和输出运算符, 操纵符也返回它所处理的流对象, 因此我们可以在一条语句中组合操纵符和数据。

我们已经在程序中使用过一个操纵符——`endl`, 我们将它“写”到输出流, 就像它是一个值一样。但 `endl` 不是一个普通值, 而是一个操作: 它输出一个换行符并刷新缓冲区。